

Étude de faisabilité

Système d'analyse vidéo et recherche de position par MCP

por **Rafael CEREZO MARTIN**
AI Engineer Openclassrooms
Aout 2026

Tabla de Contenidos

1. Architecture existante

2. Objectifs du systeme

3. Recherche technologique

3.1 Detection d'echiquier et conversion en FEN

3.2 Perimetre visuel retenu : echiquiers numeriques 2D en priorite

3.3 Model Context Protocol et FastMCP

4. Benefices et limites

4.1 Benefices attendus

4.2 Limites assumees

4.3 Criteres de qualite retenus

5. Architecture technique cible (MCP)

6. Plan structure

6.1 Decisions retenues

6.2 Etapes de developpement

6.3 Specification de la tool MCP

7. Etude de faisabilite

7.1 Choix d'infrastructure : self-hosted vs cloud

7.2 Coûts de construction (CAPEX)

7.3 Coûts recurrents (OPEX mensuel)

7.4 Scenarios de dimensionnement

8. Risques identifiés

9. Alternatives envisagees

Alternative A - Indexation cloud avec GPU manage

Alternative B - Repli par similarite de position plutot que par famille ECO

9.3 Feuille de route recommandee

10. Conclusion et prochaines etapes

Références

Abstract

Ce document presente la conception, sans implementation, d'un systeme avance de recherche de position d'echecs dans un catalogue de videos pedagogiques. Le systeme existant (Etapes 1 a 6 de la mission) recommande deja des videos YouTube pertinentes par requete textuelle sur l'ouverture jouee, mais cette approche est limitante : elle ne cible jamais un instant precis dans la video, obligeant l'utilisateur a chercher lui-meme le passage utile dans un contenu parfois long.

La solution proposee ajoute une couche d'analyse computer vision (detection d'echiquier par OpenCV, conversion en notation FEN par un modele CNN de type board-to-FEN) appliquee hors-ligne a un catalogue de videos curatees. Chaque position detectee est indexee avec son timestamp exact dans une nouvelle collection MongoDB. Cette capacite est exposee via un serveur MCP (Model Context Protocol, implemente avec FastMCP), interrogeable par l'agent LangGraph existant au moment ou une position se presente en partie.

Le systeme ne remplace pas l'existant : il ajoute une couche de precision qui s'active lorsque c'est possible, avec une degradation elegante lorsque ce n'est pas le cas.

Le document couvre successivement : l'architecture existante, les objectifs, la recherche technologique menee, les benefices et limites attendus, le schema d'architecture technique MCP, le plan structure de developpement, l'etude de faisabilite avec estimation des couts (CAPEX et OPEX), les risques identifiés et deux alternatives d'implementation.

Perimetre de l'etude

Dans le perimetre	Hors perimetre
Architecture cible et schema MCP	Developpement effectif du pipeline/MCP
Technologies justifiees, alternatives	Entrainement reel d'un modele
Estimation des couts (CAPEX + OPEX)	Constitution reelle du catalogue
Risques, limites, feuille de route	Validation juridique (droits video)

1. Architecture existante

Le POC actuel repose sur une architecture hexagonale strictement appliquee, tant cote backend (Python, FastAPI, LangGraph) que cote frontend (Angular standalone). Les composants deja operationnels sont :

- Un agent LangGraph orchestrant l'analyse de position : identification de l'ouverture (systeme ECO), consultation de la theorie (Lichess Opening Explorer avec repli sur un livre Polyglot local), evaluation par Stockfish si la position sort des sentiers battus.
- Un pipeline RAG sur Milvus, alimente par des articles Wikichess et Wikipedia, permettant une recherche contextuelle sur les familles d'ouvertures.
- Une recommandation de videos YouTube par requete textuelle (API YouTube Data v3), a partir du nom de l'ouverture identifiee.
- Une interface Angular avec echiquier interactif (ngx-chessboard) consommant l'ensemble de ces services via une API FastAPI conteneurisee (Docker Compose).

Composant existant	Evolution Etape 7
Angular / ngx-chessboard	Afficher le timestamp precis dans le lien video
FastAPI	Integrer le client MCP dans le flux existant
LangGraph	Ajouter un noeud de recherche video (client MCP)
Lichess / Stockfish / Polyglot	Inchange
Milvus	Inchange ; pas de dependance pour le FEN exact
MongoDB	Nouvelle collection position_video_index
API YouTube (Etape 4)	Conservee : ingestion + repli

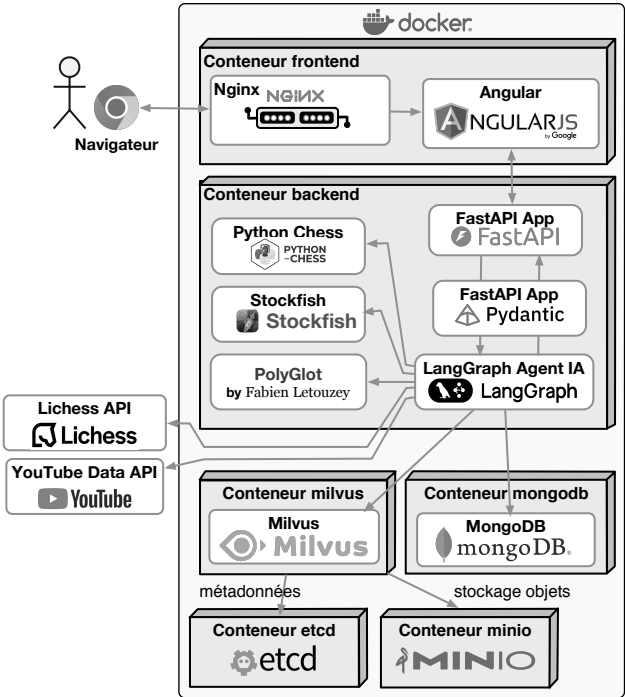


Figure : Diagramme d'Architecture existante.

C'est précisément la dernière brique - la recommandation video - que ce document se propose de faire évoluer, sans modifier les briques existantes à l'exception du graph de LangGraph.

2. Objectifs du systeme

L'objectif metier, formule par le responsable technique, est simple a enoncer mais exigeant a realiser : passer d'une recommandation video generique (un lien vers une video entiere sur une ouverture) a une recommandation precise (un lien pointant directement sur l'instant ou la position exacte de la partie en cours est expliquee).

- Objectif 1. Constituer un **catalogue de videos pedagogiques curatees**, distinct des resultats de recherche textuelle en temps reel.
- Objectif 2 - **Indexer, pour chaque video** du catalogue, les positions d'echecs qui y apparaissent, associees a **leur timestamp exact**.
- Objectif 3 - Permettre a l'agent existant **d'interroger cet index a partir de la position FEN** courante de la partie.
- Objectif 4 - **Exposer cette capacite via un protocole standard (MCP)**, reutilisable au-dela du seul agent actuel.
- Objectif 5 - **Ne jamais degrader le comportement existant** : en l'absence de correspondance, l'utilisateur recoit toujours la recommandation actuelle (Etape 4).

Pourquoi chercher par position plutot que par mot-cle

Deux arguments techniques renforcent le choix d'indexer des positions plutot que du texte.

D'abord, **les transpositions** : deux ordres de coups differents peuvent aboutir a la meme position sur l'echiquier. Une recherche textuelle sur le nom de l'ouverture ne capture pas cette equivalence, alors qu'une recherche par FEN la resout naturellement - deux parties transposees partagent le meme FEN.

Ensuite, **la continuite d'usage** : dans le systeme actuel, l'utilisateur quitte mentalement le fil de sa partie pour visionner une video generique sur YouTube. Un lien avec timestamp exact, affiche dans l'application, preserve ce contexte.

La solution proposee repose sur un pipeline en deux temps :

- **Pipeline OFFLINE (batch)** qui analyse périodiquement un catalogue de vidéos d'échecs curées, extrait des frames, détecte les positions d'échiquier via vision par ordinateur (OpenCV + CNN board-to-FEN), et indexe chaque position avec son timestamp exact.
- **Pipeline ONLINE (temps réel)** qui, lorsque le joueur effectue un mouvement, génère le FEN de la position actuelle, interroge l'index via un serveur MCP (Model Context Protocol), et renvoie instantanément le lien YouTube avec le timestamp précis.

Scenario d'usage type

Un jeune joueur réalise un coup sur l'échiquier.

La position FEN est déjà disponible dans le POC grâce à la bibliothèque chess/python-chess.

L'agent analyse la position comme aujourd'hui.

En complément, il peut demander au MCP une ressource vidéo expliquant cette position. Le système retourne les passages pertinents, idéalement classés par confiance et pertinence.

1. Le joueur effectue un coup dans Angular.
2. La position courante est validée et représentée en FEN.
3. LangGraph décide qu'une ressource vidéo est pertinente.
4. Le MCP Client appelle la tool de recherche vidéo.
5. Le MCP Server interroge l'index MongoDB.
6. Les correspondances FEN sont retournées avec URL, titre, ouverture et timestamp.
7. Angular affiche la vidéo et permet d'ouvrir directement le passage pertinent.
8. Si aucune correspondance satisfaisante n'existe, le système peut utiliser la recherche YouTube actuelle.

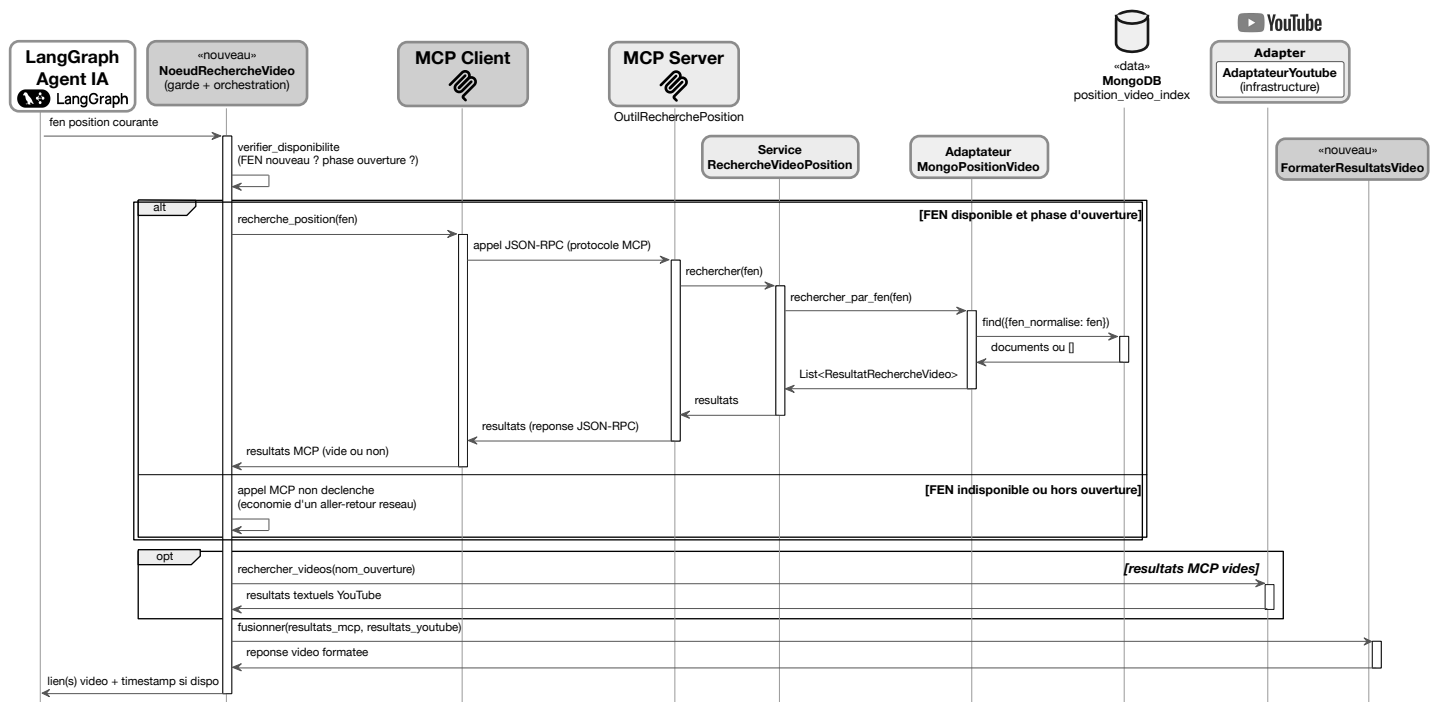


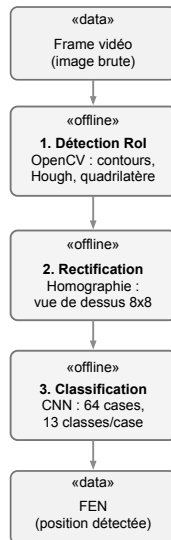
Figure: Flux en ligne. Recherche de position video (sequence, conception)

3. Recherche technologique

3.1 Detection d'echiquier et conversion en FEN

L'extraction d'un FEN à partir d'une image vidéo repose sur un pipeline classique à trois étapes :

1. **Détection de la Région d'Intérêt (Rol - Region of Interest)** : Repérage de la zone carrée du tableau à l'aide d'OpenCV (filtres de Canny, transformée de Hough pour détecter la grille de lignes, ou détecteur basé sur YOLOv8 nanodet-chess).
2. **Rectification Géométrique (Perspective Transform)** : Application d'une transformation homographique pour redresser l'échiquier vu sous angle oblique et découpage strict des 64 cases individuelles.
3. **Classification des Pièces (Board-to-FEN)** : Passage de chaque case (64 patchs d'images) dans un réseau de neurones convolutif (CNN léger type EfficientNet-B0 ou MobileNetV3) entraîné à classifier 13 classes : vide, pion (B/N), tour (B/N), cavalier (B/N), fou (B/N), dame (B/N), roi (B/N).



La conversion d'une image de plateau en notation FEN se décompose en deux étapes de nature très différente, ce qui a une conséquence directe sur les besoins d'infrastructure (section 7).

Etape A - Detection du plateau (vision classique)

La localisation du plateau dans l'image (detection de lignes par transformée de Hough, extraction des contours, homographie pour redresser la perspective) est un traitement de vision par ordinateur classique, réalisé avec OpenCV. Il ne nécessite pas de GPU : c'est un calcul léger, déterministe, qui s'exécute confortablement sur CPU.

Etape B - Classification des pieces (reseau de neurones léger)

Une fois le plateau redressé et découpé en 64 cases, un modèle de type CNN classe le contenu de chaque case (pièce et couleur, ou vide). Les implémentations open-source de référence (LiveChess2FEN, chessing2pos, fenify-3D) utilisent des architectures légères (quelques millions de paramètres), très éloignées de la taille d'un grand modèle de langage. Des benchmarks publics sur des architectures comparables (EfficientNet-B0) montrent un ratio d'environ 1,75 entre inference GPU et CPU (37,75 im/s contre 21,51 im/s) : la CPU reste pleinement utilisable, la différence est un facteur de vitesse, pas une limite fonctionnelle.

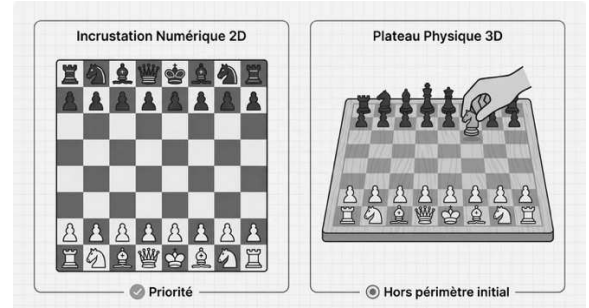
Decision retenue : le pipeline étant exécuté hors-ligne (traitement périodique du catalogue, non sollicité pendant la partie), l'absence de GPU n'est pas bloquante. Seul le temps total d'indexation du catalogue est affecté, pas la faisabilité. Voir section 7.1 pour le chiffrage.

3.2 Perimetre visuel retenu : echiquiers numeriques 2D en priorite

Une décision de périmètre réduit fortement la complexité et le risque technique du modèle de vision. Les vidéos pédagogiques d'échecs présentent généralement deux types de plateaux : des incrustations numériques 2D (superposées à l'écran par des plateformes comme

Lichess ou Chess.com, très fréquentes dans les vidéos de streaming et d'analyse), et des échiquiers physiques filmés en 3D (parties réelles, perspective oblique, mains et pièces qui masquent partiellement le plateau).

Le système cible en priorité les incrustations 2D : elles sont nettement plus faciles à détecter (pas de perspective à corriger au-delà d'un cadrage simple, pas d'occlusion par des mains, contraste net) et représentent une part importante du contenu pédagogique disponible. Les échiquiers physiques 3D restent hors du périmètre initial et sont identifiés comme extension future, avec un taux d'erreur attendu nettement plus élevé.



Solution écartée en amont : envoyer chaque frame à un modèle multimodal généraliste (type GPT-4V) plutôt que d'utiliser un CNN dédié. Écartée pour deux raisons : le coût par frame d'un LLM multimodal reste largement supérieur à celui d'un CNN léger sur de gros volumes (des centaines de milliers de frames pour quelques centaines d'heures de vidéo), et la latence par appel est incompatible avec un traitement batch efficace. Un CNN dédié, plus petit et moins coûteux à l'inférence, reste le choix pragmatique pour cette tâche de classification fermée (13 classes par case).

3.3 Model Context Protocol et FastMCP

Le MCP est un protocole ouvert permettant à un modèle ou agent d'invoquer des outils externes de manière standardisée (JSON-RPC), indépendamment du framework d'orchestration. FastMCP est l'implémentation Python retenue pour exposer le serveur, pour sa simplicité d'intégration avec un écosystème déjà basé sur FastAPI et Pydantic.

L'intérêt du MCP ici n'est pas fonctionnel (un endpoint REST classique remplirait le même rôle pour l'agent actuel) mais architectural : il découple l'accès à l'index vidéo-FEN de l'agent LangGraph spécifique de ce projet, et rend la capacité réutilisable par tout autre client compatible MCP (Claude Desktop, un futur agent de la FFE, etc.), sans coder d'intégration point à point.

Données minimales indexées

Champ	Description	Exemple
video_id	Identifiant YouTube ou identifiant catalogue	abc123
video_url	URL de la ressource	https://youtube.com/watch?v=...
title	Titre de la vidéo	Sicilian Defense explained
timestamp	Temps en secondes	742
fen	FEN détectée	r1bqk...
fen_normalized	FEN utilisée pour la recherche	r1bqk...
opening	Nom de l'ouverture	Sicilienne
eco	Code ECO si disponible	B20
confidence	Confiance du modèle de vision	0.96

Rôle du MCP

MCP ne remplace ni MongoDB ni le pipeline vidéo. Il constitue une couche d'interopérabilité entre l'agent et une capacité externe. Dans cette architecture, le LangGraph utilise un MCP Client ; celui-ci communique avec un MCP Server qui expose une tool de recherche dans l'index des positions vidéo. Le serveur peut être déployé localement ou comme service HTTP selon les besoins.

FastMCP est retenu comme option de référence pour le POC conceptuel car il permet de définir un serveur MCP en Python et d'exposer des fonctions comme tools sans implémenter manuellement la plomberie du protocole.

4. Benefices et limites

4.1 Benefices attendus

- **Precision** : passage d'un lien video generique a un lien avec timestamp exact, sur les positions couvertes par le catalogue.
- **Reutilisation de l'existant** : le systeme ECO (index_eco.json) et la validation FEN (python-chess) deja construits sont directement reutilisables pour normaliser et qualifier les positions indexees.
- **Interoperabilite** : le serveur MCP peut etre reutilise par d'autres clients futurs sans reecriture.
- **Aucune regression** : le systeme actuel (Etape 4) reste le chemin par default ; le nouveau systeme est un raffinement, pas un remplacement.

4.2 Limites assumees

- **Couverture partielle** : seules les positions effectivement presentes dans le catalogue indexe peuvent etre retrouvees avec precision. Un catalogue restreint donnera peu de correspondances exactes.
- **Recherche par FEN exact uniquement** : un repli par code ECO a ete envisage puis ecarte (section 6.1) car trop generique pour garantir un timestamp pertinent, et deja couvert par l'Etape 4.
- **Coût de maintenance du catalogue** : chaque ajout de video au catalogue necessite un nouveau passage du pipeline d'indexation.
- **Qualite variable de la detection** : angles de camera inhabituels, annotations graphiques superposees ou eclaireage degrade peuvent faire echouer la detection du plateau sur certaines frames.

Le systeme ne remplace pas l'existant : il ajoute une couche de precision qui s'active lorsque c'est possible, avec une degradation elegante lorsque ce n'est pas le cas.

Limite (section 4.2)	Où elle est traitée dans le document
Couverture partielle du catalogue	Repli automatique vers l'Étape 4, intégré par conception (section 5)
Recherche par FEN exact uniquement	Alternative B envisagée, repli par similarité (section 9), non retenue pour le POC
Coût de maintenance du catalogue	Feuille de route, curation progressive (section 9.3)
Qualité variable de la détection	Risque documenté avec probabilité/gravité/mitigation (section 8)

4.3 Criteres de qualite retenus

- Un resultat n'est jamais presente comme certain si la confiance de reconstruction FEN est insuffisante - la donnee de confiance du modele de vision accompagne chaque position indexee.
- Un timestamp doit toujours correspondre a une position reellement detectee dans la video, jamais deduit uniquement du titre ou de la description.
- Les erreurs de detection restent tracables (position rejetee, confiance faible) pour permettre une amelioration continue du pipeline, plutot que d'etre silencieusement ignorees.
- Le systeme tolere l'absence totale de resultat sans jamais bloquer la reponse a l'utilisateur.

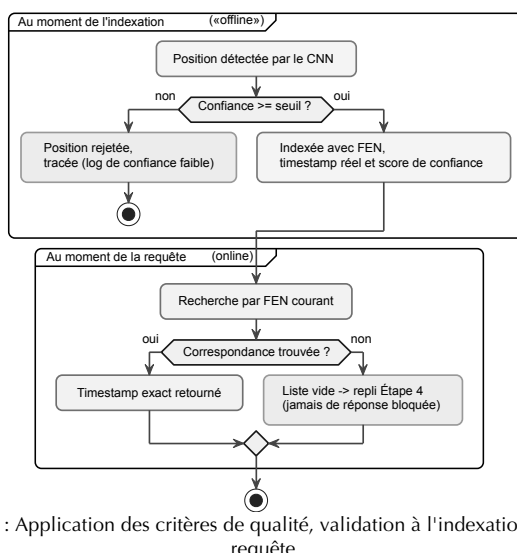


Figure : Application des critères de qualité, validation à l'indexation et à la requête.

5. Architecture technique cible (MCP)

Le système étend l'architecture microservices existante (FastAPI, LangGraph, MongoDB, Milvus, Angular) sans la perturber. Il introduit deux nouveaux conteneurs Docker :

- **Video Processing Pipeline (Worker Async)** : Service responsable de l'ingestion YouTube, de la découpe en frames, de la détection vision et du remplissage de la collection MongoDB Video_FEN_Index.
- **MCP Server Container (FastMCP)** : Serveur RPC léger exposant la fonction `seek_video_position(fen: str)` conformément à la spécification Model Context Protocol.

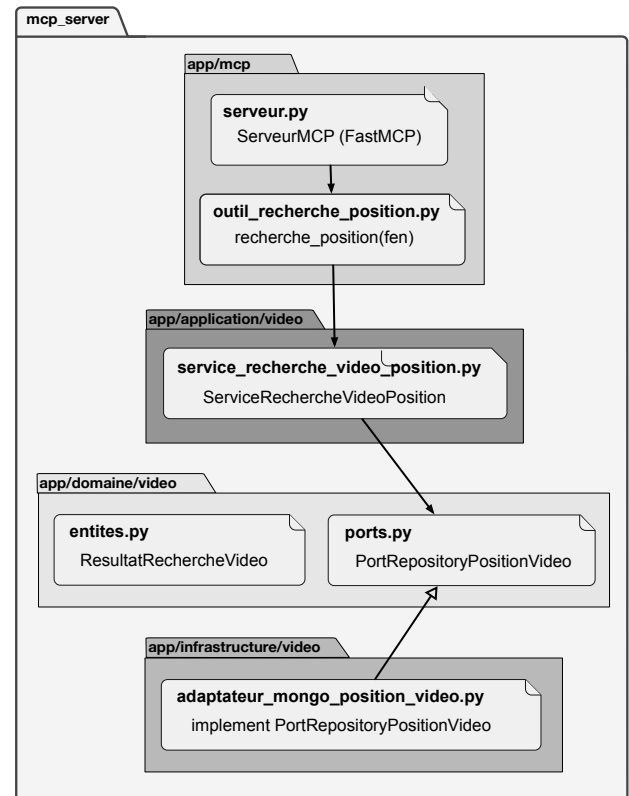


Figure : Structure interne du conteneur MCP Server (dépendances hexagonales).

La figure précédente montre la structure statique du conteneur MCP. La séquence ci-dessous illustre le même sous-système en fonctionnement, lors d'une requête réelle déclenchée par l'agent.

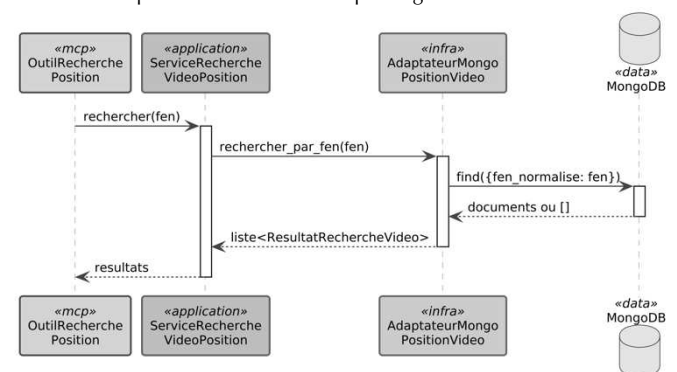


Figure : Séquence d'appel interne au conteneur MCP Server, de la tool jusqu'à MongoDB.

Le schéma suivant distingue par couleur les **composants existants** (bleu) des **composants nouveaux** (rose) : pipeline hors-ligne d'indexation, chemin de requête en ligne, serveur MCP et nouvelle collection MongoDB.

Le diagramme associe deux vues complémentaires : à gauche, **l'infrastructure Docker** telle qu'elle sera déployée (conteneurs existants en bleu, nouveaux conteneurs en jaune) ; à droite, **l'arborescence des fichiers correspondants dans le dépôt**. Les étiquettes « Étape 1 » à « Étape 5 » relient chaque bloc fonctionnel à son emplacement exact dans le code, pour faciliter la prise en main par une personne rejoignant le projet.

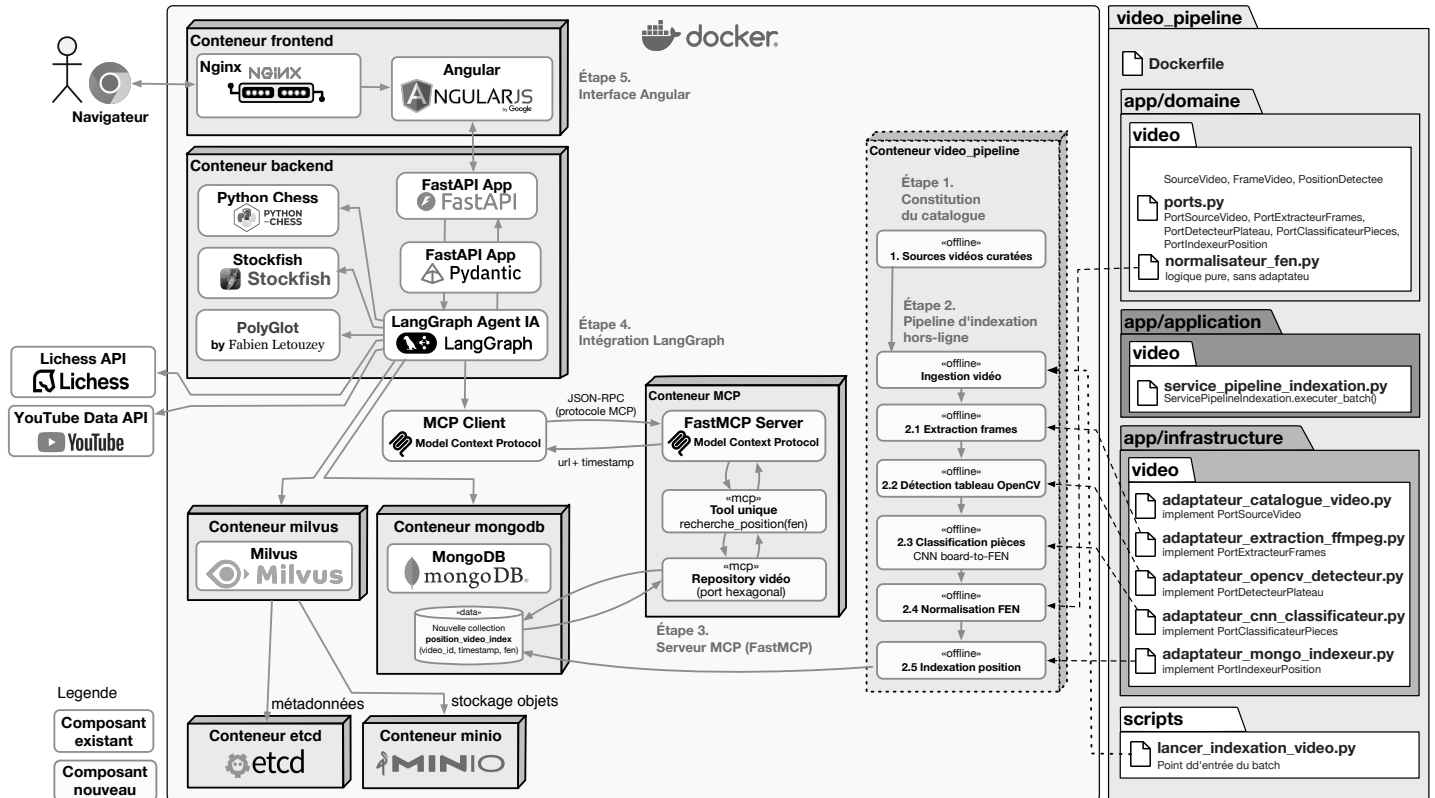


Figure : Architecture cible du système MCP vidéo infrastructure Docker et correspondance avec l'arborescence du code.

Deux flux independants composent le systeme :

- **Flux hors-ligne (traitement par lots, periodique)**
Sources videos curatees → ingestion → extraction de frames → detection du plateau (OpenCV) → classification des pieces (CNN) → normalisation du FEN → ecriture dans la collection position_video_index (video_id, timestamp, fen).
- **Flux en ligne (a chaque coup joue, cible < 500 ms)**
L'agent LangGraph transmet le FEN de la position courante a un nouveau noeud du graphe, qui invoque le serveur MCP via un client MCP. Le serveur expose une tool unique de recherche par FEN, qui interroge la collection via un repository (port hexagonal, coherent avec l'architecture existante), et renvoie l'URL et le timestamp si une correspondance existe.

Le **conteneur video_pipeline** se distingue des conteneurs backend et MCP par sa nature : il ne s'agit pas d'un service persistant à l'écoute de requêtes, mais d'un traitement par lots déclenché ponctuellement (à la demande ou par planification). Cette séparation évite d'alourdir l'image du backend, qui répond en temps réel aux coups joués, avec les dépendances lourdes de la vision par ordinateur (OpenCV, modèle CNN), réservées au seul conteneur d'indexation.

Decision de conception documentee : une seule tool MCP (recherche par FEN exact) a ete retenue plutot que deux tools separees (FEN + repli par ECO). Le repli par ECO a ete ecarte car un code ECO, meme precis, correspond a une famille de positions bien plus large qu'une position unique - il ne peut pas garantir un timestamp pertinent, et cette recherche generique est deja assuree par l'Etape 4 existante (YouTube API). Ce choix limite volontairement la surface de l'API et evite une duplication de logique.

Ces deux flux ne partagent aucune dépendance directe : ils communiquent uniquement via la collection MongoDB, ce qui permet de les faire évoluer indépendamment l'un de l'autre.

6. Plan structure

6.1 Decisions retenues

- **Infrastructure self-hosted (CPU)** pour le pipeline hors-ligne, sans dependance a un GPU cloud pour le POC (voir section 7.1).
- **Une seule tool MCP** de recherche par FEN exact, sans repli par ECO au sein du MCP.
- **Stockage de l'index dans une nouvelle collection MongoDB**, reutilisant l'infrastructure de base de donnees deja deployee.
- Le systeme existant (Etape 4, YouTube API) reste le **comportement par defaut en cas d'absence de correspondance**.

6.2 Etapes de developpement

Les etapes ci-dessous sont ordonnees par dependance. Les etapes 1 et 2 peuvent etre menees en parallele de l'etape 3 : le pipeline d'indexation ne depend pas du serveur MCP pour etre developpe et teste.

Etape	Nom	Contenu	Parallelisation
1	Constitution du catalogue	Selection des sources videos (chaines reconnues, curation manuelle). Aucune dependance technique amont.	Parallelisable avec 3
2	Pipeline d'indexation hors-ligne	Extraction de frames, detection OpenCV, classification CNN, normalisation FEN, ecriture MongoDB.	Parallelisable avec 3
3	Serveur MCP (FastMCP)	Implementation du serveur, de la tool unique et du repository d'accès a la collection.	Depend de la structure de donnees definie en 2
4	Integration LangGraph	Nouveau noeud du graphe + client MCP cote backend, branchement du fallback vers l'Etape 4.	Depend de 3
5	Interface Angular	Affichage du lien video avec lecteur positionne au timestamp exact.	Depend de 4

Étape 1. Constitution du catalogue

Sources envisagées

- **Chaînes YouTube reconnues**, orientées analyse d'ouvertures et pédagogie d'échecs
- **Playlists thématiques organisées par ouverture** (ex. guides dédiés à une défense ou un système précis)
- **Vidéos officielles de la FFE** : cours, analyses de parties commentées

Processus de curation

- Le catalogue conserve, pour chaque vidéo, ses métadonnées (titre, chaîne, durée, langue) et son état de traitement (à traiter / en cours / indexée / échec)
- La politique de sélection privilégie les chaînes, langues, durées et niveaux pédagogiques pertinents pour les jeunes espoirs visés par la mission FFE
- La curation peut être semi-automatique : proposition via l'API YouTube Data (recherche par mots-clés d'ouverture), validation manuelle par un référent avant ajout définitif au catalogue

Livrable de cette étape : une liste de vidéos identifiées (ID YouTube, URL, métadonnées), prête à être traitée par le pipeline d'indexation (Étape 2).

Étape 2. Pipeline d'indexation hors-ligne

Cette étape se décompose en 4 sous-étapes séquentielles, exécutées en batch (voir section 7.1 pour la justification du choix CPU/self-hosted).

2.1 Extraction de frames

- **Échantillonnage temporel fixe** : 1 frame extraite toutes les 2 secondes
≈ 900 frames pour une vidéo de 30 minutes
- **Raffinement possible** : détection de changement de scène (analyse d'histogramme), qui réduit le volume de frames à traiter de l'ordre de 60 à 70 % en écartant les frames quasi identiques
- **Recommandation** : commencer par l'échantillonnage simple sur le pilote, n'introduire la détection de scène que si le volume du catalogue le justifie

2.2 Détection du plateau (OpenCV)

Technique de vision classique, sans deep learning, en 5 étapes :

1. Conversion en niveaux de gris + réduction du bruit (filtre gaussien)
2. Détection de contours (Canny) + détection de lignes droites (transformée de Hough)
3. Recherche d'un quadrilatère convexe avec 4 intersections proches de 90°
4. Homographie de perspective : redressement en vue de dessus carrée (8x8)
5. Filtrage par ratio hauteur/largeur proche de 1.0 (élimine les faux positifs)

Coût estimé : de l'ordre de 10 à 50 ms par frame en CPU pur. cette étape sert de filtre grossier, éliminant rapidement les frames sans échiquier avant le traitement plus coûteux de l'étape suivante.

2.3 Classification board-to-FEN (CNN)

- **Architecture envisagée** : CNN léger (type ResNet-18, EfficientNet-B0 ou MobileNetV3)
- **Entrée** : image du plateau déjà redressé (256x256 ou 512x512)
- **Sortie** : état des 64 cases, 13 classes par case (vide, ou une des 6 pièces × 2 couleurs)
- **Taille du modèle** : de l'ordre de 2 à 5 millions de paramètres. à titre de comparaison, sans commune mesure avec un grand modèle de langage
- **Précision attendue** : 95-98 % par case sur des vidéos de bonne qualité ; >99 % de FEN complète correcte après un post-traitement de vérification de légalité (via python-chess, déjà présent dans le projet)
- **Exécution** : viable en CPU (≈21 im/s sur un Ryzen 7), un GPU ne fait qu'accélérer (≈37 im/s sur une RTX 3060, facteur ≈1,75). cohérent avec la décision self-hosted de la section 7.1

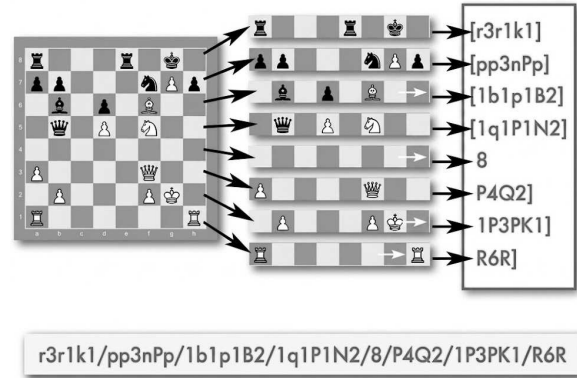


Figure : Classification board-to-FEN

2.4 Normalisation et indexation

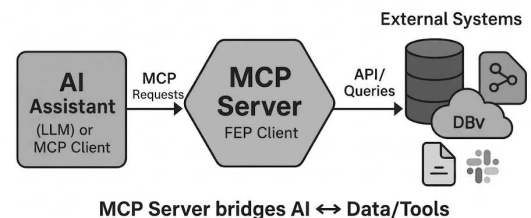
- La FEN brute contient 6 champs ; seuls les 4 premiers (position des pièces, trait, roque, en-passant) sont pertinents pour la recherche par position. les compteurs de coups ne changent pas la position géométrique
- Schéma du document stocké dans position_video_index :


```
{
  "video_id": "abc123",
  "url": "<https://youtube.com/watch?v=abc123>",
  "timestamp_secondes": 245,
  "fen": "rnbqkbnr/pppppppp/8/8/4P3/8/PPPP1PPP/RNBQKBNR",
  "fen_normalise":
    "rnbqkbnr/pppppppp/8/8/4P3/8/PPPP1PPP/RNBQKBNR b Kqkq e3",
  "eco": "B20",
  "confidence": 0.96,
  "titre_video": "Sicilian Defense: Najdorf Variation",
  "date_indexation": "2026-08-14T10:00:00Z"
}
```
- Index recommandé : sur fen_normalise, pour une recherche exacte performante

Étape 3. Serveur MCP (FastMCP)

Connexion client-serveur

Le client MCP, intégré au conteneur Backend, communique avec le serveur MCP via une connexion stdio (process local) ou HTTP+SSE (server-sent events, pour un déploiement en service distinct). Ce choix est un détail d'implémentation à trancher au moment du développement effectif. il n'affecte pas l'architecture logique décrite en section 5.



Spécification de la tool exposée

Champ	Valeur
Nom	recherche_position(fen: str)
Entrée	fen : chaîne FEN normalisée (4 premiers champs)
Sortie	Liste de {video_url, timestamp_secondes, titre_video, eco} ou liste vide
Erreurs	FEN invalide → erreur de validation ; absence de correspondance → liste vide (pas une erreur)
Protocole	JSON-RPC via MCP

Sécurité et robustesse

- Validation stricte du FEN en entrée avant toute requête MongoDB
- Rate limiting côté serveur, pour éviter qu'un usage anormal du client ne surcharge l'index

⚠ Point volontairement écarté : le document source (issu d'une proposition comparée) exposait **deux tools**. une recherche par FEN et une seconde buscar_apertura_video par code ECO, avec un paramètre eco_filter optionnel sur la première. Ce n'est **pas repris ici**, car cela contredit la décision déjà actée (section 6.1) : une seule tool, recherche par FEN exact uniquement. Le repli par ECO est volontairement laissé au système existant (Étape 4, YouTube API), pour éviter une duplication de logique. Voir la note de décision en section 5 du document pour le raisonnement complet.

Étape 4. Intégration LangGraph

Nouveau nœud et garde

- Le nouveau nœud du graphe (**rechercher_position_video**) applique d'abord une garde avant tout appel réseau au MCP, pour éviter une consultation inutile.
- Critères déclenchant l'appel effectif : la position correspond à une ouverture identifiée (code ECO présent) **et** la partie est encore en phase d'ouverture (nombre de coups faible, typiquement inférieur à 15)

Logique de décision et fallback

- Le client MCP, une fois la garde franchie, interroge la tool unique avec le FEN courant
- Si le MCP retourne un résultat : celui-ci est transmis avec son timestamp exact
- Si le MCP retourne une liste vide (ou si la garde a bloqué l'appel) : le système bascule automatiquement sur le comportement existant (Étape 4, recherche YouTube par texte). jamais de réponse bloquée

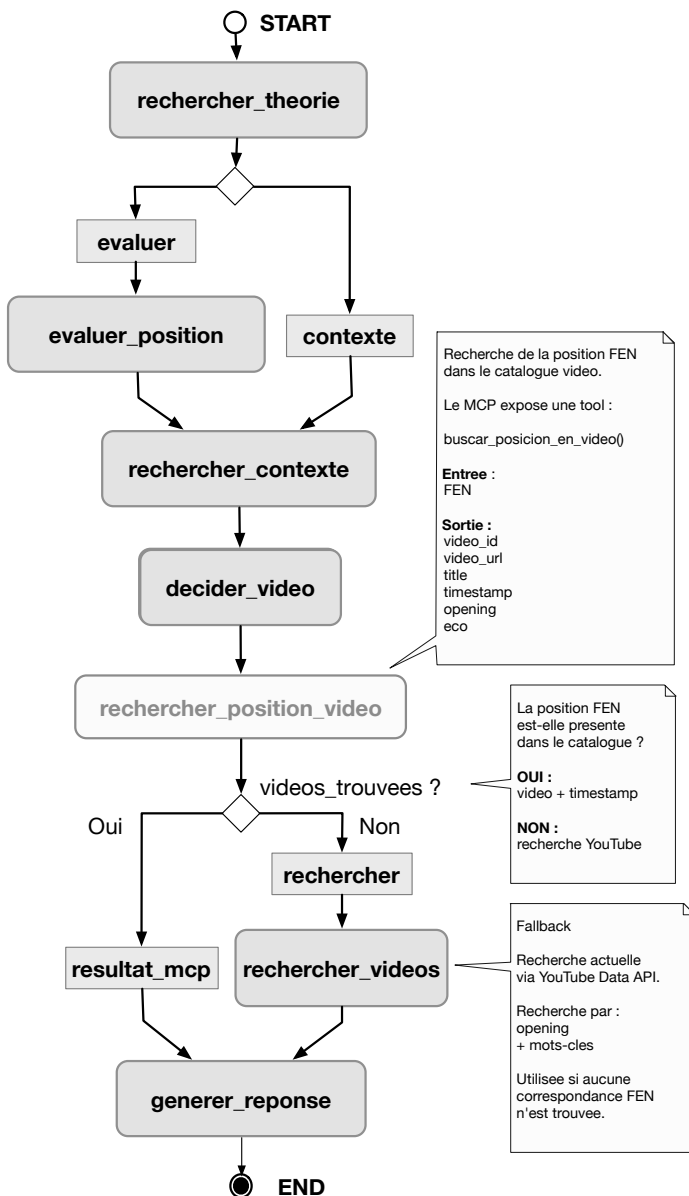


Figure : Graphe LangGraph étendu, intégration du nœud de recherche vidéo MCP.

Le contour rose identifie le seul nœud réellement nouveau du graphe (**rechercher_position_video**) ; tous les autres nœuds, y compris **rechercher_videos**, qui sert désormais de repli, proviennent du graphe existant et ne sont pas modifiés dans leur logique interne. La décision **videos_trouvees ?** ne constitue pas un nouveau branchement métier : elle traduit simplement, au niveau du graphe, le comportement déjà décrit en section 6.1, à savoir une seule tool MCP interrogée une fois, avec repli automatique si la liste retournée est vide.

Étape 5. Interface Angular

- La génération du FEN courant est déjà en place (python-chess, POC existant). aucun changement nécessaire côté capture de coup
- Le panneau de recommandations existant est **enrichi**, pas reconstruit : ajout de l'affichage du timestamp et d'un lien qui positionne le lecteur directement au bon instant
- Exemple de ce que l'interface reçoit et affiche :


```

[
  {
    "video_url": "<https://youtube.com/watch?v=abc123&t=245s>",
    "timestamp_secondes": 245,
    "titre": "Sicilian Defense: Najdorf - 6.Bg5",
    "eco": "B90"
  }
]
      
```
- Si l'origine est le MCP → timestamp exact affiché ; si l'origine est le repli YouTube (Étape 4) → lien générique, sans timestamp précis (cohérent avec la nature du système de repli)



Figure : Maquette de l'interface Angular enrichie, panneau de recommandations avec vidéo et timestamp.

6.3 Specification de la tool MCP

A titre de documentation technique pour toute personne rejoignant le projet, la tool exposée par le serveur MCP est spécifiée ci-dessous (conception, non implémentée).

Champ	Valeur
Nom	recherche_position(fen: str)
Entree	fen : chaîne FEN normalisée (4 premiers champs) de la position courante
Sortie	Liste de résultats {video_url, timestamp_secondes, titre_video, eco} ou liste vide
Erreurs	FEN invalide -> erreur de validation ; absence de correspondance -> liste vide (pas une erreur)
Protocole	JSON-RPC via MCP (FastMCP)

7. Etude de faisabilite

7.1 Choix d'infrastructure : self-hosted vs cloud

Conformement a l'infrastructure **Docker Compose** deja en place (self-hosted, sans dependance a un fournisseur cloud), le pipeline hors-ligne est concu pour fonctionner entierement en CPU.

Ce choix est justifie techniquement en section 3.1 : la detection de plateau (OpenCV) ne sollicite jamais de GPU, et la classification CNN reste viable en CPU pour un traitement par lots sans contrainte de latence.

La formule ci-dessous permet d'estimer le **temps total d'indexation du catalogue en fonction de son volume**, a affiner une fois un modele CNN concret selectionne :

$$\text{frames_a_traiter} = \frac{\text{heures_catalogue} \times 3600}{\text{secondes_par_frame_echantillonnee}}$$

$$\text{rames_par_heure_cpu} = \text{fps_cpu_estime} \times 3600$$

$$\text{heures_indexation} = \frac{\text{frames_a_traiter}}{\text{rames_par_heure_cpu}}$$

Estimation de la Durée d'Indexation du Catalogue

Exemple de calcul pour un catalogue initial de 500 heures (Production initiale).

Donnée / Formule	Résultat (exemple : catalogue de 500h)
heures_catalogue (entrée)	500 h
secondes_par_frame_echantillonnee (entrée, décision de conception)	2 s
temps_detection_par_frame (OpenCV, borne pessimiste, section 3.1)	50 ms
temps_classification_par_frame = 1000 / 21 (CNN, 21 im/s CPU, section 3.1)	47,6 ms
fps_cpu_estime = 1000 / (temps_detection_par_frame + temps_classification_par_frame)	1000 / (50 + 47,6) = 10,2 fps
frames_a_traiter = heures_catalogue × 3600 / secondes_par_frame_echantillonnee	500 × 3600 / 2 = 900 000 frames
frames_par_heure_cpu = fps_cpu_estime × 3600	10,2 × 3600 = 36 878 frames/heure
heures_indexation = frames_a_traiter / frames_par_heure_cpu	900 000 / 36 878 = 24,4 heures
Équivalent si le batch tourne 8h/jour	24,4 / 8 ≈ 3,1 jours

Risque de scalabilite identifie et accepte : si le catalogue croit fortement, le temps d'indexation augmente en CPU. La mitigation prevue n'est pas de re-architecturer le systeme, mais d'ajouter ponctuellement du calcul GPU (cloud ou local) pour les campagnes d'indexation volumineuses, sans changer l'architecture logicielle.

7.2 Coûts de construction (CAPEX)

Le choix **self-hosted** (section 7.1) ramene le cout d'infrastructure a un niveau quasi nul, mais le cout reel du CAPEX reste le temps d'ingenierie necessaire pour construire le pipeline, le serveur MCP et leur integration. Le tableau ci-dessous chiffre cet effort en jours-homme, avec un TJM indicatif fourni a titre d'ordre de grandeur (a remplacer par le cout interne reel selon le contexte de la FFE).

Poste	Effort indicatif	Notes
Pipeline hors-ligne (ingestion, OpenCV, CNN, normalisation, indexation)	12-16 j.h	Profil data / ML engineer
Serveur MCP (FastMCP, tool unique, repository)	4-6 j.h	Reutilise les conventions hexagonales existantes
Integration LangGraph + client MCP + noeud de garde	3-5 j.h	Profil backend Python

Evolution Angular (affichage timestamp)	2-3 j.h	Modification localisee, pas de refonte
Tests, observabilite, documentation technique	5-7 j.h	Inclut la doc pour prise en main par un tiers
TOTAL BUILD ESTIME	26-37 j.h	≈ 1,5-2 mois avec un developpeur

A titre indicatif seulement (TJM hypothetique de 450 EUR/jour, hors charges), cet effort correspondrait a environ 11 700-16 600 EUR de build. Ce chiffre n'est pas un devis : il sert uniquement a situer l'ordre de grandeur du projet pour la decision Go/No-Go.

7.3 Coûts récurrents (OPEX mensuel)

Deux scenarios sont presentes : le scenario retenu (self-hosted, coherent avec l'infrastructure actuelle) et un scenario cloud de reference, utile pour situer l'ordre de grandeur si le catalogue devait croitre au point de justifier une infrastructure greee.

Scenario retenu : self-hosted

Poste	Description	Cout estime
Calcul CPU (indexation periodique)	Reutilisation de l'infrastructure Docker existante	0 EUR
Stockage MongoDB (nouvelle collection)	Metadonnees uniquement (FEN, timestamp, URL) - quelques Mo par heure de video	Negligeable
Hebergement du serveur MCP	Conteneur Docker supplementaire sur l'infrastructure existante	0 EUR
Maintenance du catalogue	Ajout periodique de nouvelles videos (temps humain de curation)	Hors budget infra

Scenario de reference : escalade cloud (si volume important)

Chiffres reels au 14 aout 2026, fournis a titre de repere si le catalogue devait dépasser la capacite self-hosted actuelle.

Ressource	Ordre de grandeur	Source
MongoDB Atlas, cluster dedie M10	≈ 0,08 USD/heure ≈ 57-58 USD/mois (compute seul, hors stockage/transfert)	MongoDB Atlas, tarifs publics
GPU cloud a la demande (NVIDIA T4)	≈ 0,50-0,55 USD/heure a la demande ; ≈ 0,15-0,20 USD/heure en instance spot	Google Cloud / comparateurs GPU cloud

Un traitement ponctuel de 100 heures de GPU (par exemple pour resorber un pic de nouveaux ajouts au catalogue) couterait donc de l'ordre de 15 a 55 USD en instance spot a a peine plus de 50 USD en on-demand - une depense ponctuelle facilement justifiable si elle divise significativement le temps d'indexation, sans jamais devenir un cout recurrent obligatoire.

7.4 Scenarios de dimensionnement

A titre indicatif, l'ordre de grandeur du temps d'indexation total varie fortement selon la taille du catalogue - raison pour laquelle une montee en charge progressive est recommandee plutot qu'un chargement massif initial.

Scenario	Taille du catalogue	Temps d'indexation (ordre de grandeur)
Pilote	≈ 50h de video	Quelques heures en CPU
Production initiale	≈ 500h de video	De l'ordre de quelques jours en CPU
Echelle moyenne	≈ 2000h de video	Plusieurs jours a semaines en CPU ; GPU ponctuel pertinent

Ces ordres de grandeur restent approximatifs (ils dependent du taux de detection reel et du modele CNN retenu) et devront etre affines des le pilote (voir feuille de route, section 9).

8. Risques identifiés

Risque	Probabilite	Gravite	Mitigation
Couverture insuffisante du catalogue	Elevee	Moyenne	Fallback systematique vers l'Etape 4 existante (deja en place par conception)
Echec de detection du plateau sur certaines frames	Moyenne	Faible	Filtrage qualite en amont, echantillonnage de plusieurs frames par segment video
Temps d'indexation trop long si le catalogue grossit (videos tres longues)	Moyenne	Moyenne	Echantillonnage adaptatif, detection de changement de scene, parallelisation du batch ; option GPU ponctuelle si besoin (section 7.1)
FEN non normalise entrainant des faux negatifs de recherche	Moyenne	Elevee	Etape de normalisation FEN dediee dans le pipeline (4 premiers champs uniquement)
Derive du modele CNN sur des styles video non representes a l'entrainement	Faible	Moyenne	Selection d'un modele open-source deja valide sur des sources variees ; evaluation avant mise en catalogue
Droits d'utilisation des contenus YouTube (conditions d'utilisation, droits d'auteur)	Moyenne	Elevee	Utilisation exclusive de l'API officielle YouTube Data, jamais de telechargement ou scraping direct ; seules les metadonnees (FEN, timestamp, URL) sont stockees, pas les videos elles-memes ; validation juridique avant mise en production reelle

Le risque le plus significatif pour la valeur metier du systeme est le premier (couverture insuffisante), inherent a tout catalogue curate manuellement au demarrage. Il est mitige par construction : le systeme est additif, jamais bloquant.

9. Alternatives envisagees

9.1 Alternative A - Indexation cloud avec GPU manage

Externaliser le pipeline d'indexation vers un service cloud avec instances GPU (a la maniere du sous-systeme de vision d'un projet compare, type Azure Computer Vision + ML Endpoints).

- **Avantage** : temps d'indexation reduit d'un facteur proche de 2.
- **Inconvenient** : introduit une dependance a un fournisseur cloud, un cout recurrent, et une complexite operationnelle absente du reste du projet, qui est entierement self-hosted. Non retenue pour le POC, envisageable si le catalogue devient tres volumineux.

9.2 Alternative B - Repli par similarite de position plutot que par famille ECO

Plutot que d'ecarter completement un mecanisme de repli (section 6.1), une variante consisterait a chercher, au sein du seul catalogue deja indexe, la position la plus proche de la position courante (distance de Hamming sur le FEN, ou embedding vectoriel via Milvus) plutot qu'une correspondance exacte. Cela conserverait un timestamp approximatif mais pertinent, sans dupliquer la recherche generique de l'Etape 4. Non retenue pour la premiere version afin de garder une seule tool MCP simple, mais identifiee comme extension naturelle.

9.3 Feuille de route recommandee

Sequence recommandee, independamment du decoupage en etapes/worktrees de la section 6, pour une mise en oeuvre progressive et verifiable a chaque jalon :

1. Cadrer le catalogue (sources, criteres de qualite, droits, volume cible du pilote)
2. Constituer un jeu de validation representatif (frames annotees manuellement)
3. Prototyper la detection de plateau et la reconstruction FEN sur ce jeu restreint
4. Mesurer precision, temps par frame et taux d'echec avant d'engager le volume complet
5. Construire l'index video/FEN/timestamp (MongoDB)
6. Developper le serveur MCP et sa tool unique
7. Brancher le client MCP au graphe LangGraph existant
8. Enrichir l'interface Angular avec le timestamp
9. Tester le repli vers l'Etape 4 et mesurer la valeur percue par l'utilisateur
10. Decider ensuite : CPU permanent, GPU ponctuelle, ou architecture hybride (section 7.4)

10. Conclusion et prochaines etapes

Le systeme propose repond a la demande initiale (recommander un instant precis dans une video, pas seulement une video entiere) en s'appuyant sur des briques deja maitrisees par l'equipe (architecture hexagonale, FEN, MongoDB, systeme ECO) et sur des technologies open-source matures pour la partie vision par ordinateur. Le choix self-hosted, coherent avec le reste de l'infrastructure du projet, ramene le cout d'infrastructure a un niveau quasi nul pour le perimetre du POC, au prix d'un temps d'indexation plus long qu'une solution cloud - compromis juge acceptable et documente.

Les prochaines etapes, hors perimetre de ce document de conception, seraient : la selection concrete d'un modele CNN board-to-FEN a evaluer, la constitution d'un premier catalogue restreint (5 a 10 videos) pour valider le pipeline de bout en bout, puis l'implementation du serveur MCP decrit en section 5.

Références

Documentation technique

- Model Context Protocol, spécification officielle : <https://modelcontextprotocol.io>
- FastMCP, documentation : <https://gofastmcp.com/getting-started/welcome>
- python-chess, documentation : <https://python-chess.readthedocs.io>
- OpenCV, documentation : <https://docs.opencv.org>
- Lichess API, Opening Explorer : <https://lichess.org/api>
- YouTube Data API v3, documentation : <https://developers.google.com/youtube/v3>

Implémentations de référence citées (section 3.1)

- fenify-3D : <https://github.com/notnil/fenify-3D>
- LiveChess2FEN et chessimg2pos, cités à titre de comparaison architecturale pour le choix du CNN léger (section 3.1)

Sources

- Tarifs MongoDB Atlas officiels (<https://www.mongodb.com/pricing>),
- Page officielle de tarification GPU de Google Cloud Compute Engine (<https://cloud.google.com/products/compute/gpus-pricing>).
Les tarifs GPU varient sensiblement selon le fournisseur cloud et la région (de 0,06 à plus de 4 USD/heure selon les comparateurs multi-fournisseurs) ; les valeurs retenues ici correspondent au tarif à la demande publié par Google Cloud pour un T4 sur instance N1, à titre de référence représentative et non de devis contractuel.